Wingman: Technical Overview

Eric Hou and Julian Juan

August 9, 2026

CONTENTS

Wingman: technical overview

A plain-language map of how the system fits together, what is actually built, and what is only designed. If you read one document to understand the codebase, read this one.

---

1. What the product is

An iOS app where a person's AI agent learns them from their own ChatGPT history, recommends people worth meeting, and helps with messages after a match. The person swipes and decides. The agent never acts on their behalf.

The rule everything else is built around:

> **No fact about you reaches anyone else until you approve that specific field.**

That is not a policy written in a doc. It is enforced by a type (§4) and asserted by tests.

---

2. The pieces

Piece	What it is	Where
iOS app	SwiftUI. Onboarding, swipe deck, profile, memories, privacy.	`App/WingmanApp/`
Core library	Models, consent, matching, reply logic. Pure Swift, no UI.	`App/Shared/WingmanCore/`
Design system	Colors, type, components, the app mark.	`App/Shared/WingmanDesignSystem/`
Messages extension	Optional. Inserts a draft or intro card into iMessage.	`App/WingmanMessages/`
Gateway	Node + SQLite. Holds approved profiles, proxies model calls.	`gateway/`
Render Workflow	Fans out three reply tones in parallel to NVIDIA NIM.	`render/workflow/`
Mac companion	CLI that reads local iMessage history under Full Disk Access.	`Sources/WingmanHistory*/`

The core library is a Swift package target, so it compiles and runs from the command line without a simulator. That is why the test suite is fast and why most logic lives there rather than in views.

---

3. How a session flows

```
sign in (email)
   |
   v
pick an agent  ──► ChatGPT selected? ──► import screen ──► agent card
built
   |                                                          |
   v                                                          v
swipe deck of candidates  ◄──────────────────────────────────────
   |
   +── like ──► mutual? ──► Meet: what the agents noticed, both approve
   |
   +── stuck on a reply? ──► draft helper (3 tones, editable)
```

Routing is state-driven, not navigation-driven. `WingmanState.onboardingRoute` is a computed property with four cases, and `RootView` switches on it:

```
not signed in                    -> .login
signed in, import pending        -> .importingChatGPT
signed in, setup incomplete      -> .connectAgent
otherwise                        -> .app
```

This matters practically: the ChatGPT import used to be a modal sheet and would silently fail to appear, because it was being presented in the same update cycle as the model mutation that re-rendered its parent. Making it a route removed the race and made the step survive an app relaunch.

---

4. The privacy boundary (the important part)

Every person has one profile object with every field on it, plus a set of approved fields:

```
struct HumanProfile {
  var displayName, bio, communicationStyle: String
  var values, interests, lifestyle, boundaries: [String]
  var approvedFields: Set<ProfileField>

  var publicSnapshot: ApprovedProfile { /* emits ONLY approved fields */
}
}
```

`publicSnapshot` is the only thing any network path is allowed to serialize. Unapproved fields are not redacted downstream, they are never constructed in the first place. A field you have not ticked has no representation outside your device.

Three layers depend on this and none of them can bypass it:

1. **Sync** sends `publicSnapshot`, never `HumanProfile`.
2. **Matching** compares two `ApprovedProfile` values, never the full objects.
3. **ChatGPT-derived suggestions** land as *proposals*, outside the profile entirely, until approved.

A self test asserts the third one directly: agent-proposed interests, values, and bio must not appear in a sync payload. If someone wires a shortcut later, that test fails.

---

5. ChatGPT context: how it works

There is no API for this. OpenAI publishes nothing that reads a consumer account's conversation history. The Platform API is a separate product and cannot see chatgpt.com. The supported route is the user's own data export, requested from ChatGPT settings and emailed as a `.zip`.

This is the same wall iMessage hits (Apple exposes no transcript enumeration to a Messages extension), and it gets the same answer: an explicit, revocable, local-first grant.

The pipeline has four stages, specified in full in `CHATGPT_CONTEXT_CRAWL.md`:

Stage	What happens	Built?
1. Ingest	Read `user.json` for the account email, stream `conversations.json`.	Designed
2. Linearize	Walk each conversation tree and keep only the user's own turns.	Designed
3. Derive	Deterministic signals, then a model pass over *only* those signals.	Designed
4. Review	Everything lands unapproved, behind the per-field gate.	**Built**

THE ONE DETAIL THAT MATTERS MOST

Inside `conversations.json`, a conversation's `mapping` is a **tree, not a list**. Every edit and every regeneration forks a branch, and all branches persist forever. Iterating `mapping.values()`, which is the obvious implementation, pulls in abandoned drafts and superseded prompts, which then get counted as real signal.

The correct walk starts at `current_node` and follows `parent` pointers to the root, then reverses:

```
node := conversation.current_node
while node != nil:
    prepend node to path
    node := mapping[node].parent
```

That reconstructs the single branch the person actually saw. Then drop assistant turns, hidden system nodes, and tool artifacts.

DETERMINISTIC VS PROPOSED

Stage 3 keeps two tiers strictly apart:

- **Deterministic**: topic counts, recurring names, cadence, and custom instructions copied verbatim. Reproducible from the export alone, no model involved.
- **Proposed**: a model pass that sees *only* the deterministic summary, never raw conversation text. Because of that, a prompt injection sitting in someone's chat history has nothing to reach.

Both tiers write one object, `ChatGPTContextPacket`. That struct is the contract between the crawler and the app. Today `DemoAccounts.swift` constructs packets literally; the real crawler will decode the same struct from a file. Nothing downstream can tell the difference, which is the point: swapping in real data is a data change, not a code change.

---

6. Model calls

Reply drafting has two paths and a floor:

```
app ──► Render gateway ──► Render Workflow ──► NVIDIA NIM (3 tones in
parallel)
    └─► direct NIM  (when no gateway URL is configured)
    └─► deterministic local templates  (when anything above fails)
```

The gateway exists so the Render and NVIDIA keys never ship inside the app. Every model path falls back to local templates, so a dead network degrades quality instead of breaking the feature.

---

7. Storage

What	Where	Notes
App state	`UserDefaults` in a shared App Group	Shared with the Messages extension
Approved profiles	SQLite via the gateway	Only `publicSnapshot` fields
iMessage history	Local NDJSON on the Mac	Never syncs; manual import only

App state is versioned. `SharedStateStore.load()` accepts any schema at or below the current version and migrates it forward, and refuses only newer ones. An earlier version required an exact match, which meant a schema bump silently wiped saves the

decoder could already read.

---

8. Built vs designed

Being precise about this, because the specs are more advanced than the code.

Built and working

- Onboarding: sign in, agent picker, ChatGPT import screen, routing between them
- Per-field approval enforced by `publicSnapshot`, covered by tests
- Swipe deck with like, pass, and rewind, plus an “it’s a match” reveal animation
- Reply drafting through Render + NIM with deterministic fallback
- Profile sync gateway (Node + SQLite) over the local network
- Mac companion: access check, watch daemon with checkpointing, relationship graph builder
- Messages extension that inserts into the compose field and cannot send
- 12 core self tests

Designed, not built

- The recommender (`RECOMMENDER.md`): reciprocal rehearsal, distillation into a fast retriever
- Vector retrieval and hard eligibility filters
- Agent-to-agent micro-dialogue, and the *evidence-backed* Match Reveal transcript where every line cites the approved field it came from. The current match screen is the celebration beat, not that transcript.
- Accounts, age gate, server-backed swipes and matches, real-time chat
- The ChatGPT ingest itself (stages 1 to 3 above)

The swipe deck mixes a fixed `DemoRoster` of filler profiles with the real candidate, so there is something to swipe during a demo. Only the real one can actually match, and `SwipeCandidate.isRealCandidate` marks the difference.

The ChatGPT context currently comes from two fixed packets keyed to `eric@gmail.com` and `julian@gmail.com`. Any other address resolves to the first account, so the flow never dead-ends on a typo.

---

9. Running it

```
# core logic, no simulator needed
env DEVELOPER_DIR=/Applications/Xcode.app/Contents/Developer swift run
wingman-core-selftest

# generate and open the Xcode project
xcodegen generate --spec project.yml
open Wingman.xcodeproj
```

The package and the Xcode project must build with the same toolchain. If `swift run` reports `module compiled with Swift 6.3.3 cannot be imported`, the `.build` cache was produced by Xcode's toolchain and your PATH `swift` is older; prefix with `DEVELOPER_DIR` as above.

Mac history tools:

```
swift run wingman-history access-check
swift run wingman-history watch --acknowledge-sensitive-data
swift run wingman-history build-graph
```

Raw exports contain private messages. They are gitignored and must never be uploaded.

---

10. Where to look next

Question	Document
What is the product supposed to be?	`WINGMAN_PRODUCT_SPEC.md`
How does the recommender work?	`RECOMMENDER.md`
How does ChatGPT context get in?	`CHATGPT_CONTEXT_CRAWL.md`
Why a Mac companion for iMessage?	`MESSAGE_HISTORY_ARCHITECTURE.md`
How do I run the gateway?	`PROFILE_SYNC.md`, `RENDER_WORKFLOW.md`
What is the current state of work?	`CLAUDE_HANDOFF.md`